

Chaînes (Guide de programmation C#)

Visual Studio 2010

Une chaîne est un objet de type [String](#)¹ dont la valeur est du texte. En interne, le texte est stocké comme une collection en lecture seule séquentielle d'objets [Char](#)². Il n'y a aucun caractère Null de fin à la fin d'une chaîne C# ; par conséquent, une chaîne C# peut contenir n'importe quel nombre de caractères Null incorporés ('\0'). La propriété [Length](#)³ d'une chaîne représente le nombre d'objets **Char** qu'elle contient, et pas le nombre de caractères Unicode. Pour accéder aux divers points de code Unicode d'une chaîne, utilisez l'objet [StringInfo](#)⁴.

string et System.String

En C#, le mot clé **string** est un alias de [String](#)¹. Par conséquent, **String** et **string** sont équivalents et vous pouvez utiliser la convention d'affectation de noms que vous préférez. La classe **String** propose diverses méthodes pour créer, manipuler et comparer des chaînes en toute sécurité. De plus, le langage C# surcharge des opérateurs pour simplifier les opérations de chaînes courantes. Pour plus d'informations sur le mot clé, consultez [string \(référence C#\)](#)⁵. Pour plus d'informations sur le type et ses méthodes, consultez [String](#)¹.

Déclaration et initialisation de chaînes

Vous pouvez déclarer et initialiser des chaînes de différentes façons, comme l'illustre l'exemple suivant :

```
// Declare without initializing.
string message1;

// Initialize to null.
string message2 = null;

// Initialize as an empty string.
// Use the Empty constant instead of the literal "".
string message3 = System.String.Empty;

//Initialize with a regular string literal.
string oldPath = "c:\\Program Files\\Microsoft Visual Studio 8.0";

// Initialize with a verbatim string literal.
string newPath = @"c:\Program Files\Microsoft Visual Studio 9.0";

// Use System.String if you prefer.
System.String greeting = "Hello World!";

// In local variables (i.e. within a method body)
// you can use implicit typing.
var temp = "I'm still a strongly-typed System.String!";

// Use a const string to prevent 'message4' from
```

```
// being used to store another string value.
const string message4 = "You can't get rid of me!";

// Use the String constructor only when creating
// a string from a char*, char[], or sbyte*. See
// System.String documentation for details.
char[] letters = { 'A', 'B', 'C' };
string alphabet = new string(letters);
```

Notez que vous n'utilisez pas l'opérateur `new`⁶ pour créer un objet `String` sauf au moment d'initialiser la chaîne au moyen d'un tableau de caractères.

Initialisez une chaîne avec la valeur de constante `Empty`⁷ pour créer un objet `String`¹ dont la chaîne est de longueur zéro. La représentation sous forme de littéral de chaîne d'une chaîne de longueur zéro est `""`. En initialisant les chaînes avec la valeur `Empty`⁷ au lieu de la valeur `null`⁸, vous pouvez réduire les risques de survenue d'une `NullReferenceException`⁹. Utilisez la méthode `IsNullOrEmpty(String)`¹⁰ statique pour vérifier la valeur d'une chaîne avant d'essayer d'y accéder.

Immuabilité des objets chaîne

Les objets chaîne sont *immuables* dans le sens où ils ne peuvent pas être modifiés une fois qu'ils ont été créés. Tous les opérateurs C# et méthodes `String`¹ qui semblent modifier une chaîne retournent en fait les résultats dans un nouvel objet chaîne. Dans l'exemple suivant, lorsque le contenu de `s1` et celui de `s2` sont concaténés pour former une chaîne unique, les deux chaînes d'origine ne sont pas modifiées. L'opérateur `+=` crée une nouvelle chaîne qui contient le contenu combiné. Ce nouvel objet est assigné à la variable `s1` et l'objet d'origine qui a été assigné à `s1` est libéré pour le garbage collection parce qu'aucune autre variable n'y fait référence.

```
string s1 = "A string is more ";
string s2 = "than the sum of its chars.";

// Concatenate s1 and s2. This actually creates a new
// string object and stores it in s1, releasing the
// reference to the original object.
s1 += s2;

System.Console.WriteLine(s1);
// Output: A string is more than the sum of its chars.
```

Parce qu'une « modification » de chaîne est en fait une création de chaîne, vous devez être prudent lorsque vous créez des références aux chaînes. Si vous créez une référence à une chaîne, puis « modifiez » la chaîne d'origine, la référence continue à pointer sur l'objet d'origine, et non sur le nouvel objet créé lorsque la chaîne a été modifiée. Le code suivant illustre ce comportement :

```
string s1 = "Hello ";
string s2 = s1;
```

```
s1 += "World";

System.Console.WriteLine(s2);
//Output: Hello
```

Pour plus d'informations sur la création de nouvelles chaînes basées sur des modifications telles que les opérations de recherche et de remplacement sur la chaîne d'origine, consultez [Comment : modifier du contenu de chaîne \(Guide de programmation C#\)](#)¹¹.

Littéraux de chaîne ordinaires et textuels

Utilisez les littéraux de chaîne ordinaires lorsque vous devez incorporer des caractères d'échappement fournis par C#, comme l'illustre l'exemple suivant :

```
string columns = "Column 1\tColumn 2\tColumn 3";
//Output: Column 1      Column 2      Column 3

string rows = "Row 1\r\nRow 2\r\nRow 3";
/* Output:
    Row 1
    Row 2
    Row 3
*/

string title = "\"The \u00C6olean Harp\"", by Samuel Taylor Coleridge";
//Output: "The Æolean Harp", by Samuel Taylor Coleridge
```

Utilisez les chaînes textuelles pour des raisons de commodité et pour une meilleure lisibilité lorsque le texte de la chaîne contient des barres obliques inverses, par exemple dans les chemins d'accès des fichiers. Parce que les chaînes textuelles conservent les caractères de retour à la ligne dans le texte de la chaîne, elles peuvent être utilisées pour initialiser des chaînes multi lignes. Utilisez des guillemets anglais doubles pour insérer des guillemets anglais dans une chaîne textuelle. L'exemple suivant présente des utilisations courantes des chaînes textuelles :

```
string filePath = @"C:\Users\scoleridge\Documents\";
//Output: C:\Users\scoleridge\Documents\

string text = @"My pensive SARA ! thy soft cheek reclined
    Thus on mine arm, most soothing sweet it is
    To sit beside our Cot,...";
/* Output:
My pensive SARA ! thy soft cheek reclined
    Thus on mine arm, most soothing sweet it is
    To sit beside our Cot,...
*/

string quote = @"Her name was ""Sara.""";
//Output: Her name was "Sara."
```

Séquences d'échappement de chaîne

Séquence d'échappement	Nom du caractère	Encodage Unicode
\'	Guillemet simple	0x0027
\"	Guillemet double	0x0022
\\	Barre oblique inverse	0x005C
\0	Null	0x0000
\a	Alerte	0x0007
\b	Retour arrière	0x0008
\f	Saut de page	0x000C
\n	une nouvelle ligne	0x000A
\r	Retour chariot	0x000D
\t	Tabulation horizontale	0x0009
\U	Séquence d'échappement Unicode pour les paires de substitution	\Unnnnnnnnn
\u	Séquence d'échappement Unicode	\u0041 = "A"
\v	Tabulation verticale	0x000B
\x	Séquence d'échappement Unicode semblable à « \u », sauf au niveau de la longueur variable	\x0041 = "A"

Remarque

Lors de la compilation, les chaînes textuelles sont converties en chaînes ordinaires comportant les mêmes séquences d'échappement. Par conséquent, si vous consultez une chaîne textuelle dans la fenêtre Espion du débogueur, vous voyez les caractères d'échappement ajoutés par le compilateur, pas la version textuelle de votre code source. Par exemple, la chaîne textuelle @"C:\files.txt" apparaît dans la fenêtre Espion sous la forme "C:\\files.txt".

Chaînes de format

Une chaîne de format est une chaîne dont le contenu peut être déterminé dynamiquement pendant l'exécution. Vous créez une chaîne de format en utilisant la méthode [Format](#)¹² statique et en insérant entre accolades des espaces réservés qui seront remplacés pendant l'exécution par d'autres valeurs. L'exemple suivant utilise une chaîne de format pour sortir le résultat de chaque itération d'une boucle :

```
class FormatString
{
    static void Main()
    {
        // Get user input.
        System.Console.WriteLine("Enter a number");
        string input = System.Console.ReadLine();

        // Convert the input string to an int.
```

```

int j;
System.Int32.TryParse(input, out j);

// Write a different string each iteration.
string s;
for (int i = 0; i < 10; i++)
{
    // A simple format string with no alignment formatting.
    s = System.String.Format("{0} times {1} = {2}", i, j, (i * j));
    System.Console.WriteLine(s);
}

//Keep the console window open in debug mode.
System.Console.ReadKey();
}
}

```

Une surcharge de la méthode [WriteLine¹³](#) accepte une chaîne de format en tant que paramètre. Par conséquent, vous pouvez simplement incorporer un littéral de chaîne de format sans utiliser d'appel explicite à la méthode. Toutefois, si vous utilisez la méthode [WriteLine¹⁴](#) pour afficher la sortie de débogage dans la fenêtre **Sortie** de Visual Studio, vous devez appeler la méthode [Format¹²](#) explicitement parce que [WriteLine¹⁴](#) accepte seulement une chaîne, pas une chaîne de format. Pour plus d'informations sur les chaînes de format, consultez [Mise en forme des types¹⁵](#).

Sous-chaînes

Une sous-chaîne est une séquence de caractères contenue dans une chaîne. Utilisez la méthode [Substring¹⁶](#) pour créer une nouvelle chaîne à partir d'une partie de la chaîne d'origine. Vous pouvez rechercher une ou plusieurs occurrences d'une sous-chaîne en utilisant la méthode [IndexOf¹⁷](#). Utilisez la méthode [Replace¹⁸](#) pour remplacer toutes les occurrences d'une sous-chaîne spécifiée par une nouvelle chaîne. Comme la méthode [Substring¹⁶](#), [Replace¹⁸](#) retourne en fait une nouvelle chaîne et ne modifie pas la chaîne d'origine. Pour plus d'informations, consultez [Comment : rechercher des chaînes à l'aide de méthodes String \(Guide de programmation C#\)¹⁹](#) et [Comment : modifier du contenu de chaîne \(Guide de programmation C#\)¹¹](#).

```

string s3 = "Visual C# Express";
System.Console.WriteLine(s3.Substring(7, 2));
// Output: "C#"

System.Console.WriteLine(s3.Replace("C#", "Basic"));
// Output: "Visual Basic Express"

// Index values are zero-based
int index = s3.IndexOf("C");
// index = 7

```

Accès à des caractères

Vous pouvez utiliser la notation de tableau avec une valeur d'index pour accéder en lecture seule aux différents caractères, comme dans l'exemple suivant :

```
string s5 = "Printing backwards";

for (int i = 0; i < s5.Length; i++)
{
    System.Console.Write(s5[s5.Length - i - 1]);
}
// Output: "sdrawkcab gnitnirP"
```

Si les méthodes [String¹](#) ne fournissent pas les fonctionnalités dont vous avez besoin pour modifier les différents caractères d'une chaîne, vous pouvez utiliser un objet [StringBuilder²⁰](#) pour modifier les différents caractères « sur place », puis créer une chaîne pour stocker les résultats en utilisant les méthodes [StringBuilder²⁰](#). Dans l'exemple suivant, supposons que vous devez modifier la chaîne d'origine d'une façon particulière, puis stocker les résultats pour une utilisation ultérieure :

```
string question = "hOW DOES mICROSOFT word DEAL WITH THE cAPS LOCK KEY?";
System.Text.StringBuilder sb = new System.Text.StringBuilder(question);

for (int j = 0; j < sb.Length; j++)
{
    if (System.Char.IsLower(sb[j]) == true)
        sb[j] = System.Char.ToUpper(sb[j]);
    else if (System.Char.IsUpper(sb[j]) == true)
        sb[j] = System.Char.ToLower(sb[j]);
}
// Store the new string.
string corrected = sb.ToString();
System.Console.WriteLine(corrected);
// Output: How does Microsoft Word deal with the Caps Lock key?
```

Chaînes null et chaînes vides

Une chaîne vide est l'instance d'un objet [System.String¹](#) qui contient des caractères nuls (0). Les chaînes vides sont souvent utilisées dans divers scénarios de programmation pour représenter un champ de texte vierge. Vous pouvez appeler les méthodes sur des chaînes vides, car il s'agit d'objets [System.String¹](#) valides. Les chaînes vides sont initialisées comme suit :

```
string s = String.Empty;
```

Par contraste, une chaîne vide ne fait pas référence à l'instance d'un objet [System.String¹](#) et toute tentative d'appeler une méthode sur une chaîne vide provoque une [NullReferenceException⁹](#). Toutefois, vous pouvez utiliser les chaînes null dans les

opérations de concaténation et de comparaison avec d'autres chaînes. Les exemples suivants illustrent certains cas dans lesquels la référence à une chaîne vide provoque ou non la levée d'une exception :

```
static void Main()
{
    string str = "hello";
    string nullStr = null;
    string emptyStr = String.Empty;

    string tempStr = str + nullStr;
    // The following line displays "hello."
    Console.WriteLine(tempStr);

    bool b = (emptyStr == nullStr);
    // The following line displays False.
    Console.WriteLine(b);

    // The following line creates a new empty string.
    string newStr = emptyStr + nullStr;

    // Null strings and empty strings behave differently. The following
    // two lines display 0.
    Console.WriteLine(emptyStr.Length);
    Console.WriteLine(newStr.Length);
    // The following line raises a NullReferenceException.
    //Console.WriteLine(nullStr.Length);

    // The null character can be displayed and counted, like other chars.
    string s1 = "\x0" + "abc";
    string s2 = "abc" + "\x0";
    // The following line displays "* abc*".
    Console.WriteLine("*" + s1 + "*");
    // The following line displays "abc *".
    Console.WriteLine("*" + s2 + "*");
    // The following line displays 4.
    Console.WriteLine(s2.Length);
}
```

Utilisation de StringBuilder pour créer rapidement une chaîne

Les opérations de chaînes dans .NET sont hautement optimisées. Dans la plupart des cas, elles n'altèrent pas beaucoup les performances. Toutefois, les opérations de chaînes peuvent affecter les performances dans quelques cas tels que les boucles serrées exécutées des centaines ou des milliers de fois. La classe [StringBuilder](#)²⁰ crée une mémoire tampon de chaîne qui offre de meilleures performances si votre programme exécute un grand nombre de traitements de chaînes. La chaîne [StringBuilder](#)²⁰ vous permet également de réassigner des caractères, option que le type de données String intégré ne prend pas en charge. Par exemple, ce code modifie le contenu d'une chaîne sans créer une nouvelle chaîne :

```

System.Text.StringBuilder sb = new System.Text.StringBuilder("Cat: the ideal pet")
;
sb[0] = 'C';
System.Console.WriteLine(sb.ToString());
System.Console.ReadLine();

//Outputs Cat: the ideal pet

```

Dans cet exemple, un objet [StringBuilder](#)²⁰ est utilisé pour créer une chaîne à partir d'un jeu de types numériques :

```

class TestStringBuilder
{
    static void Main()
    {
        System.Text.StringBuilder sb = new System.Text.StringBuilder();

        // Create a string composed of numbers 0 - 9
        for (int i = 0; i < 10; i++)
        {
            sb.Append(i.ToString());
        }
        System.Console.WriteLine(sb); // displays 0123456789

        // Copy one character of the string (not possible with a System.String)
        sb[0] = sb[9];

        System.Console.WriteLine(sb); // displays 9123456789
    }
}

```